Probabilistic Algorithm with Dynamic Load Balancing for GPU-Accelerated Tumor Growth Simulations

Manuel I. Capel[1 [0000−0003−2449−4394]] and Luis Rodríguez Domingo

ETSIIT, Software Engineering Department, Universidad de Granada,
Granada 18071, Spain
{manuelcapel, luisrd}@ugr.es
https://lsi2.ugr.es/mcapel

Abstract. Efficiently simulating tumor growth using cellular automata (CA) requires high computational power. This paper presents a probabilistic CA model with a GPU–accelerated dynamic load balancing strategy implemented using CUDA, which enhances execution efficiency and scalability. The proposed approach integrates a probabilistic formulation of tumor cell transitions with adaptive GPU scheduling to avoid costly synchronization and improve parallel efficiency. Experimental results demonstrate execution time reductions of up to 54% on a 1024×1024 grid of CUDA thread blocks, while maintaining accuracy, thereby showcasing the effectiveness of our load balancing mechanism. This work bridges the gap between biologically realistic modeling and scalable GPU execution, contributing to the design of efficient computational oncology simulations.

Keywords: Cellular Automata · Tumor Growth Model · CUDA · Parallelization · Dynamic Load Balancing.

1 Introduction

Tumor growth simulation plays a crucial role in biomedical research and clinical decision-making, providing computational insight into cancer progression and treatment response. By modeling tumor growth dynamics using high-performance computing (HPC), researchers can predict tumor behavior under certain conditions, optimize medication delivery strategies, and evaluate treatment efficacy before conducting clinical trials [1].

Advances in high-performance computing (HPC) and GPU acceleration have enabled increasingly complex tumor models, making it possible to simulate large-scale, biologically realistic models that were previously computationally infeasible [19]. However, despite their biological expressiveness, cellular automata(CA)-based models remain computationally expensive. Accurate simulation of tumor growth is essential for advancing precision oncology and improving patient outcomes [2].

Despite extraordinary advances in computational modeling, tumor growth simulations still face significant challenges in terms of efficiency and scalability. Traditional mathematical approaches, based on CA [17] or reaction-diffusion differential equations, demand substantial computational power, particularly when aiming for realistic biological fidelity. Scalability issues stem from the complex interactions between tumor cells, the microenvironment, and treatment responses, which require models with high spatial and temporal resolution [13]. Furthermore, adapting these models to modern heterogeneous architectures, such as GPU clusters or distributed HPC systems, remains difficult due to inefficient load balancing, memory bottlenecks, and parallelization constraints [16]. Overcoming these challenges is essential to achieve real–time simulations that support clinical decision–making and personalized treatment design.

Parallel computing has demonstrated its effectiveness across many domains involving the simulation of complex processes, and recent advances in distributed HPC and GPU architectures have further extended this potential. Several studies have improved the efficiency of CA implementations for tumor growth modeling [9] [5]. Nonetheless, existing parallelization strategies often encounter scalability limitations due to synchronization overheads and inefficient workload distribution among processing units. In [8] and [11], the degree of parallelism had to be restricted to prevent race conditions between processes. Previous dynamic balancing approaches, such as [21], improve performance but still rely on synchronization mechanisms that limit scalability.

Other current approaches have explored computational dynamic load balancing in specific subproblem domains. For example, [10] propose techniques for efficiently executing a CA in parallel in a two-dimensional domain divided into rectangular regions, while [18] adapt load balancing to a processor grid aligned with the geometry of the problem.

In this paper, we introduce a probabilistic CA model with a GPU-accelerated dynamic load balancing mechanism that avoids costly synchronization. By updating cells independently across CUDA threads, our approach enhances scalability while preserving model accuracy. CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by NVIDIA that enables general-purpose computation on GPUs. Unlike the approach in [21], which processes each tumor cell independently but requires synchronization for shared-data update, we revisit and extend the probabilistic model in [17] to ensure that each grid cell's state is updated independently in the next iteration. This guarantees that each thread processes its own cell without requiring global synchronization or extensive atomic operations, thereby eliminating contention overhead and enabling efficient large–scale simulations.

2 Related Work

Tumor growth has been widely modeled using cellular automata (CA) due to their ability to capture complex biological behavior through simple local interaction rules [17]. In CA-based models, each cell evolves according to neighborhood-

dependent transition rules, enabling the simulation of tumor expansion, invasion, and microenvironment interactions. Unlike continuous approaches such as reaction–diffusion models [7] or systems of differential equations [15,23], CA explicitly represent individual cell dynamics and stochastic effects, making them particularly suitable for multiscale tumor modeling. Comprehensive surveys, such as [12], highlight the relevance of CA models for studying cell-cycle regulation and therapeutic response.

As model complexity increases, computational efficiency becomes a limiting factor. Parallelization techniques have therefore played a central role in enabling large-scale CA simulations. GPU-based implementations have demonstrated substantial performance improvements over CPU-based approaches. For instance, Cagigas-Muñiz et al. [3] optimized stencil-based CA simulations on CUDA architectures, reporting significant acceleration compared to sequential baselines. Similarly, Du et al. [6] introduced a GPU-powered 3D hybrid simulator capable of handling tens of millions of cells, achieving speedups of approximately 150× over parallel CPU implementations. These studies illustrate the transformative impact of GPU acceleration in computational oncology.

Load balancing remains a critical challenge in parallel CA simulations. Dynamic load balancing (DLB) strategies aim to distribute workload evenly across processing units, especially in spatially heterogeneous domains where tumor density evolves dynamically. Giordano et al. [10] proposed analytical workload redistribution methods for distributed-memory architectures, while Salguero et al. [21] developed a parallel CA tumor growth model incorporating dynamic balancing to reduce execution time. Although these approaches improve resource utilization, synchronization overhead and communication costs often limit scalability, particularly when frequent workload redistribution is required.

Despite significant progress in computational optimization, fewer works explicitly connect modeling semantics with architectural design. Most GPU-based CA studies prioritize raw throughput, often relying on deterministic update rules or simplified transition schemes to reduce synchronization costs. In contrast, probabilistic CA formulations introduce additional computational complexity but enhance biological expressiveness. Bridging the gap between probabilistic modeling and scalable GPU execution therefore remains an open methodological challenge.

Our work contributes to this line of research by integrating a probabilistic CA formulation with a GPU-oriented dynamic load balancing strategy designed to minimize synchronization overhead while preserving biological realism. By explicitly aligning modeling decisions with GPU execution constraints, the proposed approach advances both computational efficiency and methodological coherence in large-scale tumor growth simulations.

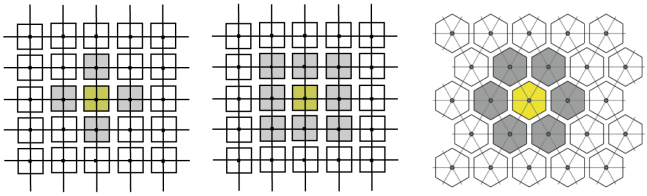


Fig. 1: Comparison of common neighborhood configurations in two-dimensional cellular automata.

3 Methodology

3.1 Tumor Growth Simulation Model

The CA model focuses on simulating tumor growth through local rules that govern the behavior of each cell according to its immediate environment. The model can be described through three fundamental components: spatial representation, behavioral rules, and underlying model assumptions.

Regarding spatial representation, the system is defined on a discrete two-dimensional grid in which each position represents either a tumor cell or a surrounding tissue element (see Figure 1). The type of neighborhood used for interaction—such as the Moore neighborhood with $\mathbf{b} = 8$ neighbors or the von Neumann neighborhood with $\mathbf{b} = 4$ neighbors—determines how each cell is influenced by its local environment. The system evolves in discrete time steps, meaning that all state changes occur synchronously at regular intervals.

Concerning behavioral rules, each cell follows a set of probabilistic mechanisms that regulate three principal parameters. The proliferation capacity (ρ_{max}) defines how frequently a cell is allowed to divide. The migration capacity (μ) determines the likelihood that a cell will move within the surrounding tissue. The spontaneous death capacity (α) specifies the probability that a cell will die without external intervention. These parameters vary depending on the cell type, particularly distinguishing tumor stem cells from tumor daughter cells.

The model further assumes that tumor growth is driven by two main cell populations. Tumor stem cells are considered immortal, characterized by unlimited proliferation ($\rho = \infty$) and zero spontaneous death probability ($\alpha = 0$), and they are capable of generating both self-renewing copies and tumor daughter cells. In contrast, tumor daughter cells have limited proliferation capacity ($\rho = \rho_{max}$) and a non-zero probability of spontaneous death ($\alpha > 0$). A key aspect of the model is competition for space. When no neighboring positions are available, a cell enters a quiescent state, becoming inactive until environmental conditions allow further activity.

Overall, this model provides a structured framework for studying tumor growth dynamics and can be efficiently optimized for high-performance computing environments through parallelization strategies implemented on GPU hardware.

3.2 GPU Parallelization Strategy

The CUDA-based implementation leverages parallel processing on Graphics Processing Units (GPUs) to accelerate tumor growth simulations. The strategy addresses three main aspects: the CUDA threading model and grid structure, memory optimization, and synchronization with race condition handling.

The computational domain is partitioned into a grid of thread blocks, where each thread is responsible for updating a single cell of the cellular automaton. Each block processes a subregion of the global grid, with threads mapped directly to individual cells. Shared memory is used within each block to temporarily store intermediate cell state updates, thereby reducing expensive accesses to global memory. The number of threads per block is carefully tuned to balance computational workload and ensure efficient memory access patterns, maximizing Streaming Multiprocessor (SM) occupancy.

From a memory optimization perspective, the implementation exploits several CUDA-specific mechanisms. Shared memory is employed to cache frequently accessed cell states locally within thread blocks, significantly reducing latency compared to global memory accesses. Global memory coalescing is enforced to align memory access patterns with warp execution, ensuring that contiguous threads access contiguous memory locations. In addition, constant memory is used to store static model parameters, minimizing redundant memory fetches and further improving bandwidth efficiency.

To guarantee correctness while maintaining performance, appropriate synchronization mechanisms are incorporated to prevent race conditions. CUDA's `_syncthreads()` primitive is used to synchronize threads within a block, ensuring consistent intermediate updates before proceeding to subsequent computation stages. Atomic operations are applied when multiple threads attempt to modify shared memory locations, thereby preserving data integrity. At a broader level, grid-level synchronization strategies are implemented to ensure coherent state updates across blocks during successive simulation steps.

The complete CUDA implementation is publicly available in our repository [4], ensuring full reproducibility. The description above summarizes the core computational strategy for clarity.

Overall, these design choices ensure data consistency across threads while maximizing parallel efficiency. The use of synchronization primitives and atomic operations effectively eliminates race conditions and preserves the integrity of cell updates. Profiling results indicate that synchronization and atomic operations account for approximately 5% of total execution time, confirming that contention overhead remains limited. As a result, the CUDA-based cellular automaton implementation achieves substantial computational acceleration, enabling large-scale tumor growth simulations to be performed within feasible time frames.

4 Computational Model and Algorithmic Approach

The tumor growth simulation follows a structured computational approach based on CA. The CUDA-based implementation leverages parallelization to efficiently model the stochastic evolution of tumor cells.

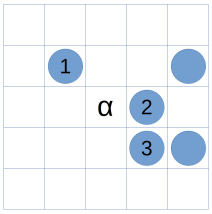


Fig. 2: Indexing of the neighbors of cell α in the Moore neighborhood.

4.1 Computational Model

The CUDA implementation follows a parallelized probabilistic approach designed to ensure efficient and consistent state updates across the two-dimensional grid. Each CUDA thread is assigned to process a single node of the grid, enabling fully parallel execution of cell state transitions. This thread-to-cell mapping ensures that the automaton evolves concurrently across the entire computational domain.

To prevent race conditions during updates, the implementation employs a double buffering strategy. Two grid states are maintained simultaneously: one stores the current configuration of the system, while the second holds the updated states computed for the next iteration. At the end of each simulation step, the buffers are swapped, ensuring that all threads operate on a consistent and immutable snapshot of the previous state.

Cancerous transformation probabilities are computed locally using the Moore neighborhood, allowing each cell's evolution to depend exclusively on its immediate surroundings. This localized computation model preserves the intrinsic spatial nature of the cellular automaton while maintaining parallel scalability. To further enhance memory efficiency, random memory accesses are minimized by enforcing an ordered neighbor processing scheme. Each neighbor is indexed systematically, beginning from the upper-left position and proceeding clockwise (Figure 2). Neighboring cells containing cancerous states contribute directly to the probability of the α cell transitioning to a cancerous state in the subsequent iteration.

Overall, this computational model combines probabilistic local interactions with GPU-oriented memory and execution optimizations, ensuring both correctness and high parallel efficiency.

4.2 Definition of the α-cell Transition Probabilities

This section considers two possible scenarios for computing the probability that the cell α contains a cancer cell in the next iteration. In the first case, α is initially non-cancerous and may transition to a cancerous state due to probabilistic influence from neighboring tumor cells. In the second case, α already contains a cancer cell and may either persist in its current state or be affected by additional contributions from neighboring cancerous cells, according to the defined transition probabilities.

Definitions and Notation Let α be a cell in a grid that may or may not contain a cancer cell. We define the following terms:

- $N(\alpha)$: the neighborhood of α (typically a Moore neighborhood, consisting of up to 8 adjacent cells in 2D).
- v: the number of neighboring cells of α that contain cancer cells.
- $V = \{a_1, a_2, \ldots, a_v\}$: the set of these cancerous neighboring cells.
- K_a: the probability that a neighboring cell $a \in V$ generates a cancer cell in α, defined as:
$$K_a = \frac{p_{mi} + p_{re}}{l_a}, \tag{1}$$
where l_a is the number of nodes in $N(\alpha)$ that do not yet contain cancer cells.
- p_{re}: the probability that a cancer cell in α persists if it is already present.
- p_{mi}: the probability that a cancer cell migrates into α from a neighboring cell.

Case 1: Probability of Cancer Cell Generation in α To compute the probability that at least one of the neighbors of α will generates a cancer cell in α, given that α does not initially contain one, we consider all possible permutations σ of the set V. For a neighbor a_0 occupying position n in the ordering σ, the probability that it is the first to generate a cancer cell in α is:

$$P(a_0, n) = \frac{1}{v!} K_{a_0} (n-1)!(v-n)! \sum_{j=1}^{\binom{n-1}{v-1}} \prod_{i=1}^{n-1} (1 - K_{\sigma(i)}). \tag{2}$$

Probability of the first neighbour generating a cancer cell in α.

Summing over all possible positions n where a_0 can generate the cell in α, we obtain the total probability:

$$P_\alpha = \sum_{a \in V} \sum_{n=1}^{v} P(a, n). \tag{3}$$

Final probability of cancerous transition in α.

This equation expresses the probability that a cancer cell arises in α due to the influence of its cancerous neighbors.

Case 2: Probability of Cancer Cell Persistence in α If α already contains a cancer cell, the probability that it will persist in the next iteration or receive a new cancer cell is:

$$P_\alpha = p_{\alpha_{re}} + (1 - p_{\alpha_{re}}) \sum_{a \in V} \sum_{n=1}^{v} P_\alpha(a, n). \tag{4}$$

Probability of α remaining cancerous or receiving a new cancer cell.

Here, $P_\alpha(a, n)$ represents the probability that the neighbor a generates a cancer cell in α at position n:

$$\begin{aligned} P_\alpha(a_0, n) = \frac{(1 - p_{\alpha_{re}})}{(v+1)!} K_{a_0} (n-1)!(v-n+1)! \\ \times \sum_{j=1}^{\binom{n-1}{v-1}} \prod_{i=\sigma(\alpha)-1}^{n-1} (1 - K_{\sigma(i)}). \end{aligned} \tag{5}$$

Here, K_{a_0} represents the probability of migration and reproduction from neighbour a_0. All possible cases where α is accessed after a_0 in the ordering are summed.

Finally, summing over all probabilities from each neighbor, we obtain the total probability formula:

$$P = \sum_{a \in V} \sum_{n=1}^{v} P_\alpha(a, n). \tag{6}$$

Final probability of cancerous transition in α.

This equation accounts for all possible contributions from neighbors, considering the order in which they are evaluated, ensuring the correct dynamics for persistence and generation of new cancer cells in α.

4.3 Algorithmic Framework and Complexity Analysis

The tumor growth simulation begins with the initialization of a two-dimensional grid representing both initial tumor cells and surrounding healthy tissue. Once initialized, the system evolves through probabilistic state update rules applied at each discrete time step. A tumor cell may proliferate with probability P_{re} provided that neighboring space is available, migrate with probability P_{mi}, or undergo programmed cell death (apoptosis) with probability P_d. These transitions are evaluated locally for each cell, preserving the spatial interaction principles of the cellular automaton.

Parallel execution is achieved through CUDA kernels, allowing all cells in the grid to be updated concurrently. Proper synchronization mechanisms are

Algorithm 1 Transition Function for Tumor Growth Simulation with CUDA

```
currentCell ← threadIdx + blockIdx ∗ blockDim
if currentCell is cancerous then
    if currentCell.nNeighbours > 0 then
        if currentCell.nNeighbours == 8 then
            nextCell ← currentCell
        else
            P ← calcProbabilities()
            r ← Random(0, 1)
            if r < P then
                nextCell ← neighbourCell
            else
                nextCell ← currentCell
            end if
        end if
    else
        nextCell ← currentCell
    end if
else
    if currentCell.nNeighbours > 0 then
        P ← calcProbabilities()
        r ← Random(0, 1)
        if r < P then
            nextCell ← neighbourCell
        else
            nextCell ← currentCell
        end if
    else
        nextCell ← currentCell
    end if
end if
```

employed to maintain data consistency across threads, ensuring that state transitions are computed from a coherent snapshot of the previous iteration. The simulation proceeds iteratively until a steady-state growth regime or predefined convergence condition is reached. This CUDA-based design significantly enhances computational efficiency, enabling large-scale tumor growth simulations to be performed with high performance and numerical stability.

The implementation relies on three specialized CUDA kernels that structure the computational workflow. The neighborhood update kernel scans the Moore neighborhood of each cell to determine the number of cancerous neighbors contributing to local transition probabilities. The transition function kernel evaluates these probabilities and determines the next state of each cell according to the defined stochastic rules. A dedicated cancer stem cell persistence kernel ensures that stem cells maintain their immortal characteristics throughout the simulation, preserving the biological assumptions of the model. The function **calcProbabilities()** described in Algorithm 1 computes the probability that

each neighboring tumor cell generates a cancer cell in α, storing these values in a vector. The specific neighboring cell responsible for inducing the transition is then selected according to the computed probability distribution. The recursive probability combination functions for both cancerous and non-cancerous cells follow the Poleszczuk model and are available in the public code repository.

From a computational perspective, the probability calculation procedure involves nested loops, recursive calls, and combinatorial operations that influence overall complexity. In the theoretical worst case, the recursive formulation leads to a complexity of $\mathcal{O}(v!)$, where v denotes the number of relevant neighboring cells. However, in practice, GPU parallelization distributes these computations across threads, reducing the effective per-thread computational cost to $\mathcal{O}(v)$. This parallel reduction in complexity makes large-scale simulations computationally feasible while preserving the probabilistic accuracy of the underlying biological model.

4.4 Experimental Setup

Simulations were performed on an NVIDIA GeForce GTX 1650 with Max-Q design, using CUDA. We tested grid sizes of 512×512 and 1024×1024 nodes used for 25-day and 50-day simulations, respectively. A single stem cell was placed at the center of the grid as the initial condition, with each simulation step representing one hour, resulting in 600 steps for 25-day simulations and 1200 steps for 50-day simulations. The performance evaluation focused on three key metrics: *execution time*, measuring the computational time required for each iteration; *speedup*, assessing the performance improvement of the GPU implementation over a sequential execution; and *load balancing efficiency*, analyzing the workload distribution across CUDA thread blocks. The sequential implementation serves as the baseline for measuring speedup. While qualitative comparisons with [21] are included, our aim is not to replicate their setup but to evaluate the proposed probabilistic model's scalability and efficiency under comparable conditions. Although execution times are slightly higher than those reported in [21], the proposed strategy provides improved scalability and more uniform GPU utilization, highlighting its potential for larger-scale simulations.

Model Parameters and Calibration Table 1 summarizes the main probabilistic parameters used in the tumor growth simulation. The proliferation rate ($\rho_{\max}$), migration probability (μ), and spontaneous death rate (α) define the biological behavior of tumor stem and daughter cells, while p_{mi} and p_{re} correspond to the migration and persistence probabilities used in the stochastic transition rules (Eqs. 1–6). Parameter values were drawn from previous studies on high-performance CA tumor models [17] and adjusted empirically to reproduce realistic growth dynamics and spatial expansion patterns consistent with [18,10]. These values balance computational feasibility with biological plausibility and were verified through stability tests of the growth curve over multiple simulation runs.

Table 1: Representative parameter values used in the probabilistic CA tumor growth model.

Parameter	Symbol	Range	Value	Description
Proliferation capacity	$\rho_{\max}$	3–6 div.	5	Max. divisions per daughter cell
Migration probability	μ	0.05–0.15	0.10	Probability of migration
Spontaneous death rate	α	0.01–0.10	0.05	Probability of apoptosis
Cell migration factor	p_{mi}	0.05–0.20	0.12	Probability of cancer cell inflow
Persistence factor	p_{re}	0.70–0.95	0.85	Probability of persistence in α

Parameter Selection and Calibration The probabilistic parameters of the model were selected using a two-stage calibration process based on the reference biological model of Poleszczuk and Enderling, as well as the experimental validation performed in the underlying study.

First, the parameters governing cell dynamics –namely, proliferation capacity (ρ_{max}), migration probability (μ), spontaneous death rate (α), migration factor (ρ_{mi}) and persistence factor (ρ_{re}) –were initialised within the biologically plausible ranges reported in previous CA tumour growth models. These ranges were chosen to reflect typical avascular tumour behaviour and spatial expansion patterns.

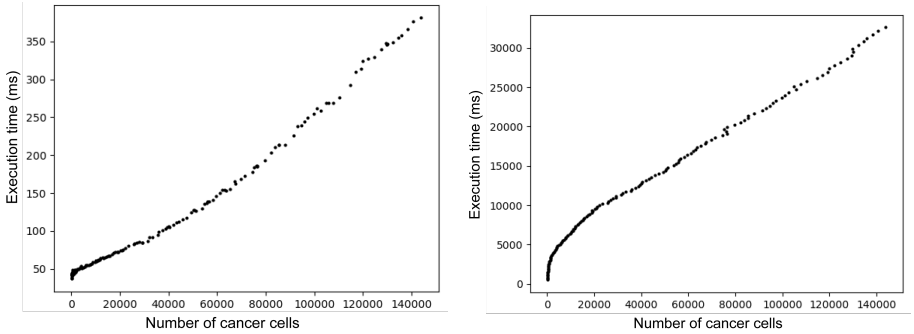
Secondly, the parameter values were empirically calibrated through repeated simulation runs to ensure that the resulting tumour growth dynamics satisfied the following criteria: (i) monotonic expansion without numerical instability; (ii) spatially compact tumour morphology with a dense core; and (iii) growth curves consistent with classical tumour growth models (logistic and Gompertz), and (iv) stability of results across multiple stochastic realisations.

Model reliability was assessed by fitting the simulated growth curves to analytical growth functions, and by evaluating the goodness of fit using standard error metrics. The selected parameter configuration corresponds to the median setting that produced stable growth behaviour and minimal fitting error across repeated experiments. The detailed experimental procedure and growth-curve validation are reported in the underlying implementation study [20].

This calibration procedure ensures that the chosen parameters strike a balance between biological plausibility, numerical stability and computational feasibility, as demonstrated by the experimental analysis of the underlying implementation.

5 Experimental Results

Since tumor growth simulations require a high resolution, we have chosen a 1024×1024 grid to represent the tumor tissue. This grid size limits the tumor to 1,048,576 cells, which are initially empty (non-cancerous tissue) at the beginning of the simulation and can be progressively occupied by tumor cells.



(a) Time per simulation step. (b) Total simulation time.

Fig. 3: Comparison between the number of cancer cells and simulation time. (a) Time per simulation step. (b) Total time for simulation.

5.1 Performance Comparison

The performance of the proposed tumor growth simulation was evaluated using various execution parameters and CUDA grid configurations. The total execution time and the number of processed cells per time unit were measured with a 1024×1024 grid over 150 simulation days. A comparison with previous work [21] highlights three key points: (a) the proposed algorithm processes a similar number of cells as in [21], demonstrating that the tumor growth model remains consistent. (b) Unlike [21], which used 4000 steps, this work uses 3600 steps to match the 150-day simulation time window, where each day consists of 24 steps, $150\,\text{days} \times 24\,\text{steps/day} = 3600$ steps. (c) Although the best execution times of the proposed approach are slightly worse that those in [21], the grid cell distribution strategy enhances scalability, which was not achieved in the previous work. This work primarily evaluates computational performance.

CUDA Grid Size and Speedup Analysis The speedup achieved when varying the CUDA grid size is detailed in Table 2. The results show a significant speedup compared to single-thread execution.

Speedup increases with grid size, reaching $218.647\times$ for a 1024×1024 configuration. The simulation achieves 8.79M processed cells/s in only 17.187s.

Execution Time Curve Analysis The relationship between number of tumor cells and execution time per step is crucial. Execution time grows approximately linearly with tumor size, with minor deviations due to increased neighborhood interactions (Figure 3a).

Similarly, Figure 3b shows that total execution time initially increases rapidly as the tumor grows, but eventually, growth stabilizes and follows an approximately linear trend. This confirms that the probabilistic model remains efficient even for larger simulations.

Table 2: Performance of the simulations carried out with the probabilistic model, varying the size of the CUDA grid.

CUDA Grid	Size	Processed	Processed/s	Seconds	Speedup
1x64	164,401	191,918,090	40,234.41	4769.998	1
2x64	136,588	146,984,801	78,683.08	1868.061	1.956
4x64	153,697	173,264,862	84,914.98	2040.451	2.111
8x64	146,279	169,838,837	147,932.54	1148.083	3,677
16x64	146,784	160,235,895	250,391.66	639.941	6.223
16x16	147,533	176,305,011	136,524.60	1291.379	3.393
32x64	137,424	154,349,732	498,481.24	309.64	12.389
32x32	158,803	180,625,230	262,416.52	688.315	6.522
64x64	141,702	161,861,208	908,587.38	178.146	22.582
128x128	119,016	123,115,379	2,344,429.65	52.514	58.269
256x256	139,710	147,640,588	4,906,306.92	30.092	121.943
512x512	129,254	142,093,466	5,581,705.07	25.457	138.729
1024x1024	134,349	151,196,626	8,797,150.52	17.187	218.647

Comparative Discussion with GPU CA Models To contextualize the performance results in Table 2, we compared our probabilistic GPU-based cellular automaton with other recent GPU implementations for CA-driven tumor or biological simulations.

Cagigas-Muñiz *et al.* [3] reported a speed-up of approximately 150× over CPU implementations for large-scale CA stencils optimized with CUDA, while Du *et al.* [6] achieved a 150-fold acceleration in their 3D hybrid simulator (*Gell*), which handles tens of millions of cells through simplified deterministic update rules. Our implementation reaches a maximum speed-up of about 219× relative to a sequential baseline for a 1024^2 grid (approximately one million cells).

Table 3: Kernel execution times and execution times examined using Nvprof.

Kernel / Operation	**Time (%)**	**Total**	**Calls**	**Avg**	**Min**	**Max**
transitionFunction()	80.76%	169.448s	3600	47.069ms	5.3988ms	175.90ms
updateNeighbors()	18.41%	38.6204s	3600	10.728ms	10.392ms	11.412ms
`CUDA memcpy DtoH`	0.77%	1.61256s	31	52.018ms	51.493ms	53.980ms
setup_kernel()	0.03%	65.804ms	1	65.804ms	65.804ms	65.804ms
`CUDA memcpy HtoD`	0.02%	51.896ms	1	51.896ms	51.896ms	51.896ms
stemCellTest(Cel)	0.01%	18.802ms	3600	5.2220µs	2.3040µs	7.3920µs

Although the absolute throughput of our simulator is roughly 40–50% of the per-cell update rate reported in [6] for comparable grid sizes, our model incorporates probabilistic transition rules and dynamic load balancing, which increase computational complexity but yield greater biological realism and scalability. Therefore, the obtained performance is consistent with the state-of-the-art GPU

CA models, demonstrating that probabilistic formulations can approach the efficiency of deterministic GPU frameworks while providing a more expressive modeling capability for tumor growth dynamics.

Profiling (Table 3) shows transitionFunction() dominates runtime ($\approx 81\%$). GPU occupancy of the function decreases to $\approx 36\%$ in later iterations, due to uneven workload distribution.

5.2 Sensitivity Analysis

A local sensitivity analysis was conducted around the calibrated parameter values to evaluate the robustness of the model and its suitability for heterogeneous biological scenarios. Sensitivity analysis is widely used in computational biology to assess model robustness and parameter identifiability under uncertainty [14]. The adopted procedure follows standard practices in local sensitivity analysis for complex computational models [22]. Each key parameter was varied independently while the others were kept fixed. (a) migration probability μ: $\pm 20\%$; (b) The spontaneous death rate (α): $\pm 20\%$; and (c) the persistence factor (ρ_{re}): $\pm 10\%$. For each configuration, 10 simulation runs were performed. Two indicators were measured: (1) the final tumour size after 50 simulated days, and (2) the average execution time per simulation step.

Table 4: Local sensitivity analysis

Parameter variation	Change in final tumor size	Change in execution time
μ -20%	-8.7%	+0.6%
μ +20%	+9.3%	+0.8%
α -20%	+11.5%	+0.4%
α +20%	-10.2%	+0.5%
ρ_{re} -10%	-14.8%	+1.1%
ρ_{re} +10%	+16.2%	+1.3%

As expected for avascular growth dynamics, results indicate that biological outcomes are moderately sensitive to persistence and death parameters, while computational performance varies by less than 1.5% in all cases.

This confirms that: (a) model behaviour is robust to moderate parameter uncertainty; (b) GPU performance is largely independent of biological parameter tuning; and (c) the proposed implementation can be adapted to different biological scenarios without a significant computational impact.

The sensitivity trends are consistent with the growth-curve stability analysis reported in the underlying experimental study.

The sensitivity methodology follows standard practices in computational biology and uncertainty analysis. These trends are consistent with the stability analysis of tumor growth dynamics reported in the experimental study [20].

5.3 Scalability Analysis

When designing a parallel GPU implementation for tumor growth simulations, the grid partitioning strategy strongly affects SM occupancy, memory behavior, and load balancing.

We evaluated two partitioning schemes: (a) division into 16 full rows or columns, and (b) division into 64 smaller regions. A hybrid strategy combining both approaches was then designed to improve overall efficiency.

In the 16-row/column scheme, workload imbalance occurs because tumor growth is concentrated near the center, leaving some SMs underutilized. This configuration also shows reduced cache efficiency and uneven execution times, since the processing cost depends on the tumor density assigned to each block.

	Average	Min	Max	Sum
SM Active Cycles	435.886.667,94	428.384.647	443.847.847	6.974.186.687
SMSP Active Cycles	428.369.846,02	399.514.675	444.196.286	27.415.670.145
L1 Active Cycles	435.886.667,94	428.384.647	443.847.847	6.974.186.687
L2 Active Cycles	410.789.433,50	410.592.168	410.975.932	3.286.315.468
DRAM Active Cycles	621.931.620	620.156.424	623.757.140	2.487.726.480

Fig. 4: Load distribution on the different SMs of the GPU for the distribution of the cell grid in 64 regions.

The 64-region scheme, Figure 4, improves load balance by distributing active cells more evenly across SMs. However, this approach increases scheduling overhead and may degrade memory locality due to more scattered access patterns.

The proposed hybrid strategy combines the memory locality of the row-based scheme with the load balancing of the region-based approach. This design ensures a more uniform workload distribution while preserving contiguous memory accesses.

Table 5: Comparison of Workload Distribution Strategies

Factor	**Comparison of Workload Distribution Strategies**		
	16 Rows/Columns	**64 Regions**	**Hybrid Grid (Expected)**
SM Utilization	× Poor	✓ Good	✓ Best
L1/L2 Cache	× Lower efficiency	✓ Better caching	✓ Optimal cache utilization
DRAM Usage	× Lower, but inefficient	✓ Higher but contemption	✓ Balanced throughput

Profiling results confirm that the hybrid strategy achieves near-uniform SM utilization. As shown in Figure 4, all SMs operate within a narrow execution range (approximately 428–443 million cycles), whereas the row-based configuration exhibits significant imbalance (not shown). In addition, improved spatial locality increases L1/L2 cache effectiveness and reduces global memory traffic, resulting in lower memory latency and better overall performance (Table 5).

6 Discussion

The proposed probabilistic model combined with hybrid load balancing improves GPU utilization and scalability, enabling efficient large-scale tumor growth simulations. Compared to row-, column-, or region-based partitioning, the hybrid strategy achieves more uniform GPU occupancy, illustrating how modeling semantics directly influence both biological realism and computational feasibility. Profiling indicates that execution time is primarily concentrated in the transition and neighborhood-update kernels, suggesting future optimizations through kernel fusion and improved memory access patterns. Overall, the approach aligns modeling methodology with scalable GPU execution, contributing to computational oncology within a feasible and fully available framework.

The probabilistic formulation increases biological expressiveness by explicitly modeling stochastic proliferation, migration, and persistence. This realism introduces additional computational overhead relative to deterministic CA, as transition probabilities must be evaluated for every cell at each iteration. The transition kernel accounts for approximately 80% of total runtime, reflecting this cost. Nevertheless, GPU parallelism sustains high throughput and scalability, demonstrating that biologically meaningful stochastic dynamics can be supported at scale. Proper architectural mapping allows enhanced realism without sacrificing performance. Correctness is ensured through intra-block synchronization using `_syncthreads()` and limited atomic operations for shared updates. These mechanisms account for roughly 5% of total runtime and remain stable across grid sizes and tumor densities, indicating that synchronization is not a scalability bottleneck. By restricting coordination to localized operations and avoiding global communication, the design minimizes contention while preserving numerical integrity. Alternative lock-free approaches would require restructuring transition semantics and increasing global memory traffic. The chosen strategy therefore balances efficiency, scalability, and implementation simplicity.

7 Conclusion & Future Work

This paper presents a probabilistic algorithm with dynamic load balancing for GPU-accelerated tumor growth simulations, achieving significant improvements in execution time, scalability, and load balancing efficiency compared to static approaches. The proposed hybrid workload distribution ensures balanced GPU utilization and enhanced computational performance. Future work will focus on extending the probabilistic model toward biologically validated simulations and incorporating adaptive balancing in multi-GPU and distributed environments to further improve scalability and applicability to complex biomedical scenarios. The C++/CUDA implementation of the tumor growth simulation code is available at [4].

References

1. Anderson, A., Chaplain, M.: Continuous and discrete mathematical models of tumor-induced angiogenesis. Bulletin of Mathematical Biology **60**(5), 857–900 (1998)
2. Begg, R., al.: Machine learning in cancer research: Applications and challenges. Nature Reviews Cancer **20**(11), 660–674 (2020)
3. Cagigas-Muñiz, D., del Rio, F.D., Sevillano-Ramos, J.L., Guisado-Lizar, J.L.: Efficient simulation execution of cellular automata on gpu. Simulation Modelling Practice and Theory **118**, 102519 (2022). https://doi.org/10.1016/j.simpat.2022.102519
4. Capel, M.I., Rodríguez Domingo, L.: Load balancing strategies for parallel tumor growth simulations. https://github.com/mcapeltu/Load-Balancing-Strategies-for-Parallel-Tumor-Growth-Simulations (2025), accessed: 2025-10-13
5. Cicirelli, F., Forestiero, A., Giordano, A., Mastroianni, C.: Parallelization of space-aware applications: Modeling and performance analysis. Journal of Network and Computer Applications **122**, 115–127 (2018)
6. Du, J., Zhou, Y., Jin, L., Sheng, K.: Gell: A gpu-powered 3d hybrid simulator for large-scale multicellular system. PLoS ONE **18** (2023)
7. Gatenby, R., Gawlinski, E.: A reaction-diffusion model of cancer invasion. Cancer Research **56**, 5745–5753 (1996)
8. Gerakakis, I., Gavriilidis, P., Dourvas, N.I., Georgoudas, I.G., Trunfio, G.A., Sirakoulis, G.C.: Accelerating fuzzy cellular automata for modeling crowd dynamics. Journal of Computational Science **32**, 125–140 (2019)
9. Giordano, A., Amelia, F., Gigliotti, S., Rongo, R., Spataro, W.: Load balancing of the parallel execution of two dimensional partitioned cellular automata. In: Proceedings of the 30th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP). pp. 205–210 (2022)
10. Giordano, A., Rango, A.D., Rongo, R., D'Ambrosio, D., Spataro, W.: Dynamic load balancing in parallel execution of cellular automata. IEEE Transactions on Parallel and Distributed Systems **32**(2), 470–484 (2021)
11. Grama, A.Y., Gupta, A., Kumar, V.: Isoefficiency: measuring the scalability of parallel algorithms and architectures. IEEE Parallel Distributed Technology: Systems and Applications **1**(3), 12–21 (1993)
12. Gérard, C., Goldbeter, A.: Computational models of the cell cycle: Past, present, and future. Nature Computational Science **4**(1), 1–12 (2023). https://doi.org/10.1038/s41540-024-00397-7
13. Kim, H., al.: Computational challenges in tumor growth modeling and simulation. IEEE Transactions on Biomedical Engineering **68**(4), 1223–1235 (2021)
14. Marino, S., Hogue, I.B., Ray, C.J., Kirschner, D.E.: A methodology for performing global uncertainty and sensitivity analysis in systems biology. Journal of Theoretical Biology **254**(1), 178–196 (2008). https://doi.org/10.1016/j.jtbi.2008.04.011
15. Padder, A., Shah, T.R., Afroz, A., et al.: A mathematical model to study the role of dystrophin protein in tumor micro-environment. Scientific Reports **14**, 1–15 (2024)
16. P.Macklin: Key challenges in multiscale modeling of cancer. Annals of Biomedical Engineering **47**, 2263–2281 (2019)
17. Poleszczuk, J., Enderling, H.: A high-performance cellular automaton model of tumor growth with dynamically growing domains (2013)

18. Rango, A.D., Giordano, A., Mendicino, G., Rongo, R., Spataro, W.: Tailoring load balancing of cellular automata parallel execution to the case of a two-dimensional partitioned domain. The Journal of Supercomputing **79**, 9273–9287 (2023)
19. Rice, J.R., al.: Accelerating multiscale tumor growth simulations using gpus. IEEE Transactions on Biomedical Engineering **65**(6), 1525–1536 (2018)
20. Rodríguez Domingo, L.: Dynamic Load Balancing Strategy for Tumor Growth Simulations on GPU Hardware. Bachelor thesis, University of Granada, Granada, Spain (2024)
21. Salguero, A.G., Capel, M.I., Tomeu, A.J.: Parallel cellular automaton tumor growth model. In: Practical Applications of Computational Biology & Bioinformatics (2018)
22. Saltelli, A., Ratto, M., Andres, T., Campolongo, F., Cariboni, J., Gatelli, D., Saisana, M., Tarantola, S.: Global Sensitivity Analysis: The Primer. Wiley, Chichester, UK (2008). https://doi.org/10.1002/9780470725184
23. Yin, A., Moes, D., van Hasselt, J., et al.: A review of mathematical models for tumor dynamics and treatment resistance evolution of solid tumors. Pharmacometrics & Systems Pharmacology **8**(9), 720–737 (2019)